Department of Electrical Engineering

Namal University Mianwali

Tourism Management System

Database Engineering Project Report: Phase 4 (Final)

Prepared by

Muhammad Usman Shah (NUM-BSEE-2022-28)

Date: 17/06/2026

Instructor: Mr. Muhammad Bilal

Abstract

This report presents the design and implementation of a Tourism Management System database with IoT based transport monitoring. The main purpose of this project is to manage tourism data in a proper and organized way. The system stores information about tourists, destinations, hotels, tour packages, bookings, payments, guides, reviews, transport records, sensor readings, alerts, trip summaries, and maintenance logs.

A manual tourism system can create many problems. These problems include duplicate records, booking mistakes, unclear payment details, poor guide assignment, and weak transport monitoring. To solve these issues, a relational database was designed using proper database engineering concepts. The project includes entity identification, relationship design, ERD development, Enhanced ERD development, database assumptions, normalization, anomaly removal, MySQL implementation, SQL queries, and Power BI visual analysis.

The ERD explains the basic structure of the database. The EERD adds advanced concepts such as specialization, aggregation, associative entities, multivalued attribute handling, and derived attributes. The database was normalized to reduce repetition and improve data consistency. Power BI was used to present selected database objectives through charts. This project provides a strong database foundation for a practical tourism management system.

Table of Contents

Abstract 2

1. Introduction 5

2. Problem Statement 5

3. Project Objectives 5

4. Scope of the System 6

5. Methodology 6

5.1 Requirement Analysis 6

5.2 Entity Identification 6

5.3 Relationship Identification 7

5.4 ERD Development 7

5.5 Enhanced ERD Development 7

5.6 Normalization 7

5.7 Anomaly Removal 7

5.8 MySQL Implementation 7

5.9 Power BI Visualization 7

6. Database Design Assumptions 7

7. Main Entities and Their Purpose 8

8. Entity Relationship Diagram 9

9. Enhanced Entity Relationship Diagram 12

10. Normalization 15

10.1 First Normal Form 15

10.2 Second Normal Form 15

10.3 Third Normal Form 15

11. Anomaly Removal 16

12. Database Implementation in MySQL 17

13. Database Verification and Display Queries 20

14. Important SQL Analysis Queries 21

15. Power BI Dashboard Objectives 27

16. Discussion 29

17. Limitations 31

18. Conclusion 31

1. Introduction

Tourism organizations deal with a large amount of data every day. They need to manage tourists, destinations, hotels, tour packages, bookings, payments, guides, reviews, and transport details. If this data is stored manually or in unstructured files, it becomes difficult to manage.

A tourist may book one or more tour packages. A package belongs to a destination. A destination may have many hotels. A guide may work on many packages. A transport vehicle may be used for different routes. The same transport can also generate IoT sensor readings during a trip.

Without a proper database, many problems can occur. Tourist records may be repeated. Booking information may become incorrect. Payments may not link clearly with bookings. Guide assignments may be missed. Destination popularity may be hard to check. Transport safety may also be weak because sensor readings and alerts are not stored properly.

A relational database solves these problems. It stores data in separate but connected tables. It reduces duplicate records. It improves data accuracy. It also makes reporting easier. In this project, a Tourism Management System database is designed and implemented using MySQL. Power BI is also used to show useful visual results from the database.

2. Problem Statement

Tourism organizations need a reliable system to store, process, and analyze tourism data. A manual or poorly designed system can create serious problems. Tourist records can be duplicated. Booking history can become unclear. Payment details may not match with the correct bookings. Guide assignment can become difficult to manage. Destination records may not show accurate demand.

Transport monitoring is also an important issue. Tourism transport needs proper tracking and safety monitoring. Without structured IoT data, the system cannot properly store speed readings, fuel levels, engine temperature, passenger count, alerts, trip summaries, and maintenance actions.

This project solves these problems by designing a complete relational database. The database connects tourism records with transport monitoring records. It supports better data storage, better reporting, and better decision making.

3. Project Objectives

1. To design a relational database for a Tourism Management System.
2. To identify the major entities, attributes, primary keys, foreign keys, and relationships.
3. To develop an Entity Relationship Diagram for the proposed database.

4. To develop an Enhanced Entity Relationship Diagram using advanced database modelling concepts.
5. To normalize the database and reduce redundancy.
6. To identify and remove possible insertion, update, deletion, duplication, and derived data anomalies.
7. To implement the database in MySQL.
8. To write SQL queries for verification and analysis.
9. To analyze selected database objectives using Power BI visualizations.

4. Scope of the System

The scope of this system includes the main areas of a tourism business. It manages tourist records, destination records, hotel information, tour packages, bookings, payments, guides, guide assignments, and reviews.

The system also includes a transport module. This module stores transport records, IoT sensor readings, transport alerts, trip summaries, and maintenance logs. This helps in route monitoring, safety tracking, and transport performance analysis.

This project focuses on database design, SQL implementation, and analytical visualization. It does not include a complete web application. It does not include mobile application development. It does not include online payment gateway integration. It also does not include real hardware deployment for IoT devices. The IoT part is designed at the database level only.

5. Methodology

5.1 Requirement Analysis

The first step was to study the basic requirements of a tourism management process. The system needs to store records about tourists, destinations, hotels, packages, bookings, payments, guides, reviews, and transport. The system also needs to store IoT data for transport monitoring.

5.2 Entity Identification

After requirement analysis, the main entities were identified. These entities include Tourists, Destinations, Hotels, TourPackages, Bookings, Payments, Guides, PackageGuides, Reviews, Transport, IoT_SensorReadings, IoT_TransportAlerts, IoT_TripSummary, and IoT_MaintenanceLog.

5.3 Relationship Identification

The relationships between entities were identified. A tourist can make many bookings. A booking belongs to one tourist. A package can have many bookings. A destination can have many packages and hotels. A package can have many guides. A guide can work on many packages. A transport record can generate many sensor readings. A sensor reading can create transport alerts.

5.4 ERD Development

The ERD was developed to show the main database structure. It shows entities, attributes, primary keys, foreign keys, and cardinalities. It helps to understand how all tables are connected.

5.5 Enhanced ERD Development

The EERD was developed to show advanced database concepts. It includes specialization, generalization, aggregation, associative entities, multivalued attribute handling, dependent entities, and derived attributes.

5.6 Normalization

Normalization was applied to reduce repeated data. It also improves consistency and accuracy. The database was checked using First Normal Form, Second Normal Form, and Third Normal Form.

5.7 Anomaly Removal

Possible anomalies were identified and corrected. These include insertion anomaly, update anomaly, deletion anomaly, duplication anomaly, cardinality anomaly, and derived data anomaly. These problems were reduced by using better table design, foreign keys, unique constraints, and controlled derived fields.

5.8 MySQL Implementation

The database was implemented in MySQL. All tables were created with suitable data types, primary keys, foreign keys, and constraints.

5.9 Power BI Visualization

Power BI was used to create visual charts from the database. These charts help to understand tourist nationality distribution, most booked destinations, and transport route utilization.

6. Database Design Assumptions

1. Each tourist can make many bookings.
2. Each booking belongs to one tourist.
3. Each booking is linked with one tour package.

4. Each tour package belongs to one destination.
5. Each destination can have many hotels.
6. Each destination can have many tour packages.
7. A tourist can write many reviews.
8. Each review belongs to one tourist and one destination.
9. A tour package can have many guides.
10. A guide can work on many tour packages.
11. PackageGuides resolves the many to many relationship between TourPackages and Guides.
12. Each transport record can generate many IoT sensor readings.
13. A sensor reading can trigger one or more transport alerts.
14. A transport alert may create a maintenance log.
15. Trip summaries are generated from IoT readings and alert records.
16. Admin is treated as a system user entity. It does not have a direct foreign key relationship with the main transaction tables.

7. Main Entities and Their Purpose

Table 1. Main entities of the Tourism Management System.

Entity	Primary Key	Entity Type	Purpose
Admin	admin_id	Strong entity	Stores system user account details.
Tourists	tourist_id	Strong entity	Stores tourist personal and contact information.
Destinations	destination_id	Strong entity	Stores destination details and ratings.
Hotels	hotel_id	Strong entity	Stores hotel details linked with destinations.
TourPackages	package_id	Strong entity	Stores package name, duration, price, and destination.
Bookings	booking_id	Dependent entity	Stores booking records made by tourists.
Payments	payment_id	Dependent entity	Stores payment details linked with bookings.
Guides	guide_id	Strong entity	Stores guide details and experience.
PackageGuides	assignment_id	Associative entity	Links guides with tour packages.

Reviews	review_id	Dependent entity	Stores tourist reviews for destinations.
Transport	transport_id	Strong entity	Stores transport and route details.
IoT_SensorReadings	reading_id	Strong entity	Stores IoT readings from transport vehicles.
IoT_TransportAlerts	alert_id	Dependent entity	Stores alerts generated from sensor readings.
IoT_TripSummary	summary_id	Strong reporting entity	Stores trip summary and safety data.
IoT_MaintenanceLog	log_id	Dependent entity	Stores maintenance actions after alerts.

Bookings, Payments, Reviews, IoT_TransportAlerts, and IoT_MaintenanceLog are treated as dependent entities. They are not strict weak entities because they have their own primary keys. However, their business meaning depends on parent entities. For example, a payment has no clear meaning without a booking. A maintenance log also has no clear meaning without a transport alert.

8. Entity Relationship Diagram

The Entity Relationship Diagram shows the basic design of the Tourism Management System database. It includes all major entities, their attributes, primary keys, foreign keys, and relationships. It also shows the cardinality between tables.

Entity Relationship Diagram

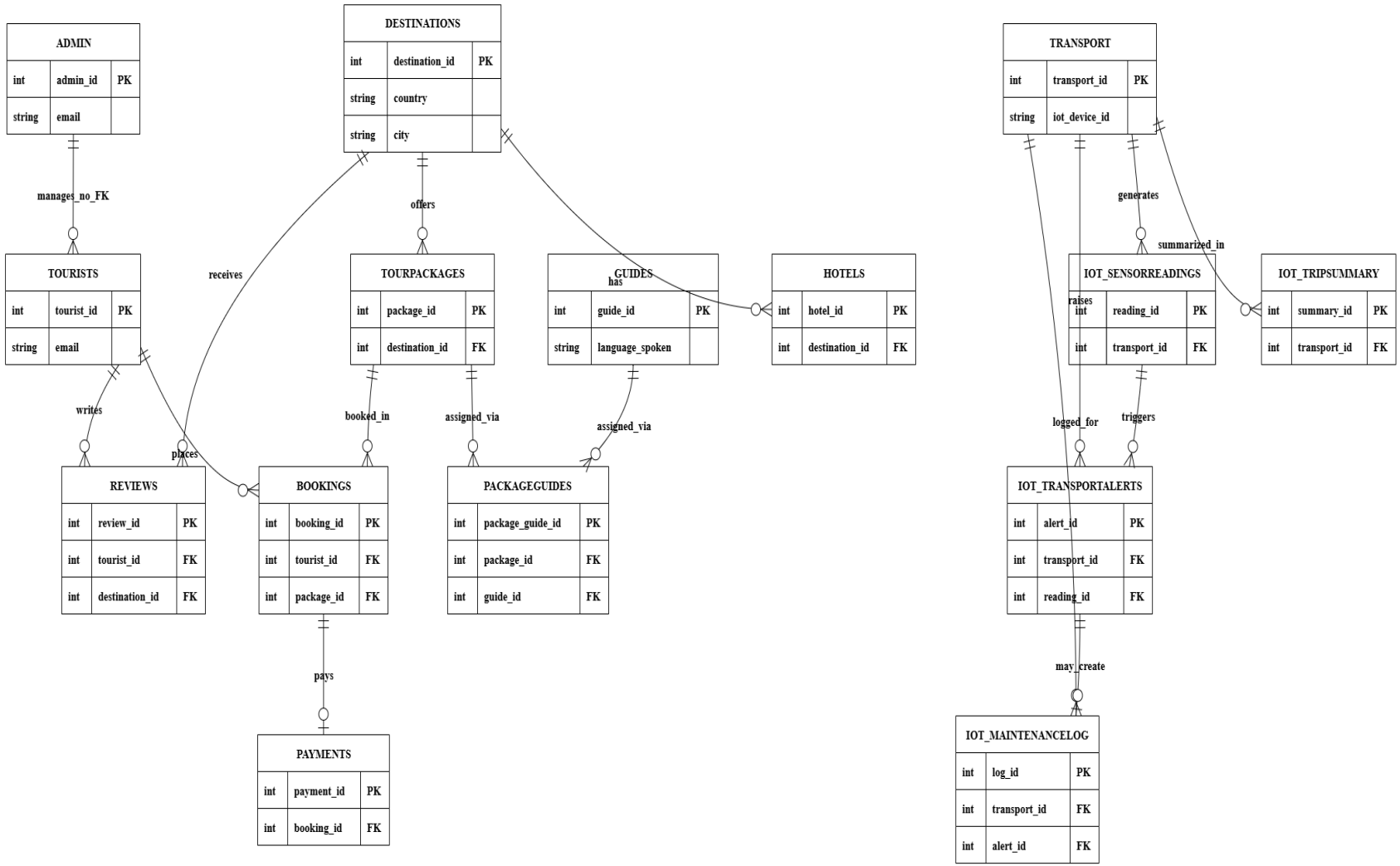


Figure 1 Entity Relationship Diagram

The ERD shows that tourists are connected with bookings. Each booking is connected with one tour package. Each tour package belongs to one destination. Destinations are connected with hotels and reviews. Guides are connected with tour packages through the PackageGuides table.

The transport part of the ERD starts from the Transport entity. Each transport record can generate many IoT sensor readings. These readings can create alerts. Alerts can lead to maintenance records. Trip summaries are stored to show the overall result of a transport trip.

Table 2. ERD relationship summary.

Relationship	Cardinality	Foreign Key	Explanation
Tourists to Bookings	1:N	Bookings.tourist_id	One tourist can make many bookings.
TourPackages to Bookings	1:N	Bookings.package_id	One package can be booked many times.
Bookings to Payments	1:1 intended	Payments.booking_id	One booking is expected to have one payment.
Tourists to Reviews	1:N	Reviews.tourist_id	One tourist can write many reviews.
Destinations to Reviews	1:N	Reviews.destination_id	One destination can receive many reviews.
Destinations to Hotels	1:N	Hotels.destination_id	One destination can have many hotels.
Destinations to TourPackages	1:N	TourPackages.destination_id	One destination can have many packages.
TourPackages to PackageGuides	1:N	PackageGuides.package_id	One package can have many guide assignments.
Guides to PackageGuides	1:N	PackageGuides.guide_id	One guide can be assigned to many packages.
Transport to IoT_SensorReadings	1:N	IoT_SensorReadings.transport_id	One transport can generate many readings.
IoT_SensorReadings to IoT_TransportAlerts	1:N	IoT_TransportAlerts.reading_id	One reading can trigger many alerts.

Transport to IoT_TransportAlerts	1:N	IoT_TransportAlerts.transport_id	One transport can have many alerts.
Transport to IoT_TripSummary	1:N	IoT_TripSummary.transport_id	One transport can have many trip summaries.
Transport to IoT_MaintenanceLog	1:N	IoT_MaintenanceLog.transport_id	One transport can have many maintenance logs.
IoT_TransportAlerts to IoT_MaintenanceLog	1:1 intended	IoT_MaintenanceLog.alert_id	One alert is expected to create one maintenance record.

9. Enhanced Entity Relationship Diagram

The Enhanced Entity Relationship Diagram extends the normal ERD. It shows advanced database modelling concepts. These concepts make the design more complete and more meaningful

Enhanced Entity Relationship Diagram

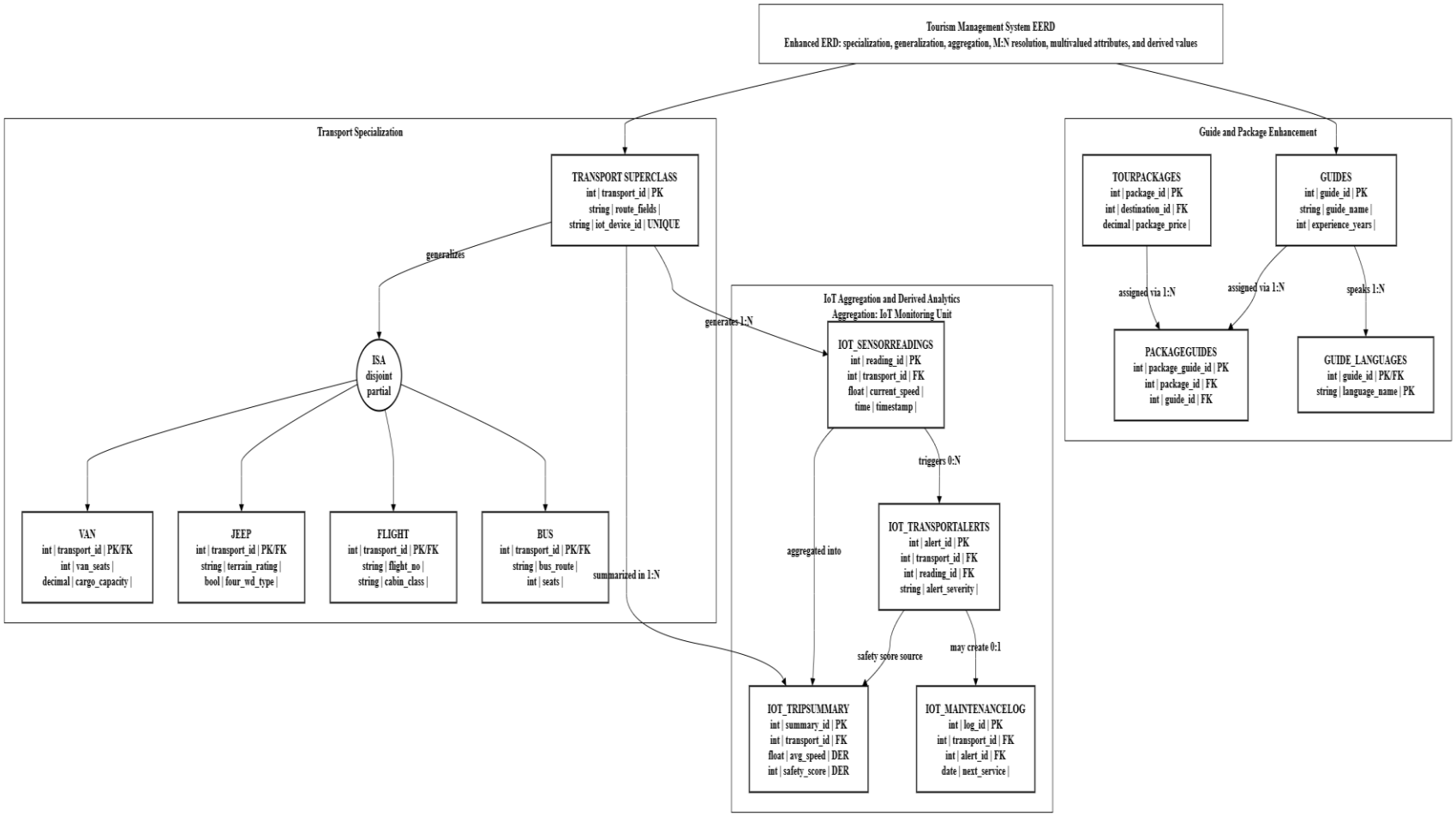


Figure 2 Enhanced Entity Relationship Diagram

In the EERD, Transport is used as a superclass. It stores common transport details such as route, company, ticket price, IoT device information, departure time, arrival location, and status. Bus, Flight, Jeep, and Van are used as subtypes. These subtypes can store details that are specific to each transport type.

The specialization is disjoint. This means one transport record belongs to only one subtype at a time. For example, a transport record cannot be both Bus and Flight at the same time. The specialization is partial. This means some transport records may remain only in the main Transport table without being placed into a subtype.

The Guides entity can have multiple languages. Storing many languages in one field is not a good design. Therefore, GuideLanguages is added to store one language per row. This removes the multivalued attribute problem.

PackageGuides is an associative entity. It resolves the many to many relationship between TourPackages and Guides. A package can have many guides. A guide can also work on many packages.

IoT monitoring is represented as an aggregation. It includes Transport, IoT_SensorReadings, and IoT_TransportAlerts. This shows that trip summary and maintenance actions depend on the whole transport monitoring process.

IoT_TripSummary stores controlled derived values. These include average speed, alert count, trip status, and safety score. IoT_MaintenanceLog records maintenance actions created from important transport alerts.

Table 3. EERD features.

EERD Feature	Application in This Database	Benefit
Specialization	Transport is divided into Bus, Flight, Jeep, and Van.	Stores type specific data clearly.
Generalization	Common transport data is stored in Transport.	Reduces repeated transport fields.
Aggregation	Transport monitoring combines transport, readings, and alerts.	Shows IoT monitoring as one process.
Associative Entity	PackageGuides connects TourPackages and Guides.	Resolves many to many relationship.
Multivalued Attribute Handling	GuideLanguages stores guide languages separately.	Satisfies First Normal Form.

Dependent Entities	Payments, Reviews, Alerts, and Logs depend on parent records.	Keeps business meaning clear.
Derived Attributes	Trip summary stores average speed and safety score.	Supports reporting and analysis.

10. Normalization

Normalization is the process of organizing database tables correctly. It helps reduce repeated data. It also improves accuracy and consistency.

10.1 First Normal Form

First Normal Form requires each field to store atomic values. This means each field should store only one value.

Most tables in this database satisfy First Normal Form. However, Guides.language_spoken can violate First Normal Form if many languages are stored in one field. For example, storing English, Urdu, and Arabic in one column is not atomic.

To solve this problem, a separate table is used:

```
GuideLanguages(guide_id, language_name)
```

This table stores one language per row for each guide. This makes the design cleaner and more correct.

10.2 Second Normal Form

Second Normal Form requires every non key attribute to depend on the full primary key. Most tables in this database use a single primary key. Therefore, they mostly satisfy Second Normal Form.

The PackageGuides table needs special attention because it connects packages and guides. The same guide should not be assigned to the same package more than once. To prevent this, the following unique constraint is used:

```
UNIQUE(package_id, guide_id)
```

This improves the design and avoids duplicate guide assignments.

10.3 Third Normal Form

Third Normal Form requires that non key attributes should not depend on other non key attributes. Some fields in this database are derived fields. These fields can create update problems if not controlled properly.

Table 4. Third Normal Form issues.

Field	Issue	Recommended Correction
Bookings.total_amount	It depends on booked price and number_of_people.	Store booked_price and calculate total amount through a controlled rule.
Destinations.rating	It can be calculated from Reviews.rating.	Use a view or controlled trigger.
IoT_SensorReadings.speed_status	It depends on speed fields.	Use a generated column or calculate during alert generation.
IoT_TripSummary.avg_speed_kmh	It is derived from sensor readings.	Store only as controlled reporting data.
IoT_TripSummary.safety_score	It is derived from alerts and trip violations.	Calculate it using a fixed safety rule.

11. Anomaly Removal

A database anomaly is a problem that can happen during insertion, update, or deletion of records. These problems make the database unreliable. The following anomalies were identified and corrected in this project.

Table 5. Anomaly removal.

Anomaly Type	Location	Problem	Correction
Insertion anomaly	Guide languages	Multiple languages in one field make insertion unclear.	Create GuideLanguages table.
Update anomaly	Destination rating	Stored rating may differ from review average.	Calculate rating from Reviews or use a trigger.
Update anomaly	Booking amount	Package price changes can confuse old booking totals.	Store booked_price at booking time.
Duplication anomaly	PackageGuides	Same guide can be assigned repeatedly to the same package.	Add UNIQUE(package_id, guide_id).
Cardinality anomaly	Payments	Multiple payments may be stored for one booking.	Add UNIQUE(booking_id) if

			one payment per booking is required.
Deletion anomaly	Bookings and Payments	Deleting a booking can damage financial history.	Use restrict rules or booking_status.
IoT summary anomaly	Trip summary	Summary can become outdated if raw readings change.	Generate summary after trip completion and protect raw readings.

These corrections improve the reliability of the database. They also make the data more consistent. The system becomes safer for long term use.

12. Database Implementation in MySQL

The database was implemented in MySQL. Each table was created with a primary key. Foreign keys were used to connect related tables. Unique constraints were used where duplicate records should not be allowed.

```
CREATE DATABASE IF NOT EXISTS TourismManagementSystem;
USE TourismManagementSystem;

CREATE TABLE Tourists (
    tourist_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(150) NOT NULL,
    nationality VARCHAR(100),
    email VARCHAR(150) UNIQUE,
    phone VARCHAR(30),
    passport_number VARCHAR(50),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Destinations (
    destination_id INT AUTO_INCREMENT PRIMARY KEY,
    destination_name VARCHAR(150) NOT NULL,
    country VARCHAR(100),
    city VARCHAR(100),
    description TEXT,
    rating DECIMAL(3,2) DEFAULT 0.00
);

CREATE TABLE Hotels (
    hotel_id INT AUTO_INCREMENT PRIMARY KEY,
    destination_id INT NOT NULL,
    hotel_name VARCHAR(150) NOT NULL,
    star_rating INT,
    price_per_night DECIMAL(10,2),
    contact_number VARCHAR(30),
    FOREIGN KEY (destination_id) REFERENCES Destinations(destination_id)
);

CREATE TABLE TourPackages (
    package_id INT AUTO_INCREMENT PRIMARY KEY,
    destination_id INT NOT NULL,
    package_name VARCHAR(150) NOT NULL,
```

```
    duration_days INT,
    price_per_person DECIMAL(10,2),
    availability_status VARCHAR(50),
    FOREIGN KEY (destination_id) REFERENCES Destinations(destination_id)
);

CREATE TABLE Bookings (
    booking_id INT AUTO_INCREMENT PRIMARY KEY,
    tourist_id INT NOT NULL,
    package_id INT NOT NULL,
    booking_date DATE NOT NULL,
    travel_date DATE,
    number_of_people INT DEFAULT 1,
    booked_price DECIMAL(10,2),
    total_amount DECIMAL(10,2),
    booking_status VARCHAR(50),
    FOREIGN KEY (tourist_id) REFERENCES Tourists(tourist_id),
    FOREIGN KEY (package_id) REFERENCES TourPackages(package_id)
);

CREATE TABLE Payments (
    payment_id INT AUTO_INCREMENT PRIMARY KEY,
    booking_id INT NOT NULL UNIQUE,
    payment_date DATETIME,
    payment_amount DECIMAL(10,2),
    payment_method VARCHAR(50),
    payment_status VARCHAR(50),
    transaction_reference VARCHAR(100),
    FOREIGN KEY (booking_id) REFERENCES Bookings(booking_id)
);

CREATE TABLE Guides (
    guide_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(150) NOT NULL,
    email VARCHAR(150),
    phone VARCHAR(30),
    experience_years INT,
    specialization VARCHAR(100)
);

CREATE TABLE GuideLanguages (
    language_id INT AUTO_INCREMENT PRIMARY KEY,
    guide_id INT NOT NULL,
    language_name VARCHAR(100) NOT NULL,
    FOREIGN KEY (guide_id) REFERENCES Guides(guide_id)
);

CREATE TABLE PackageGuides (
    assignment_id INT AUTO_INCREMENT PRIMARY KEY,
    package_id INT NOT NULL,
    guide_id INT NOT NULL,
    assignment_date DATE,
    UNIQUE(package_id, guide_id),
    FOREIGN KEY (package_id) REFERENCES TourPackages(package_id),
    FOREIGN KEY (guide_id) REFERENCES Guides(guide_id)
);

CREATE TABLE Reviews (
    review_id INT AUTO_INCREMENT PRIMARY KEY,
    tourist_id INT NOT NULL,
    destination_id INT NOT NULL,
    rating DECIMAL(3,2),
    review_text TEXT,
    review_date DATE,
    FOREIGN KEY (tourist_id) REFERENCES Tourists(tourist_id),
```

```
    FOREIGN KEY (destination_id) REFERENCES Destinations(destination_id)
);

CREATE TABLE Transport (
    transport_id INT AUTO_INCREMENT PRIMARY KEY,
    transport_type VARCHAR(50),
    company_name VARCHAR(150),
    departure_location VARCHAR(150),
    arrival_location VARCHAR(150),
    ticket_price DECIMAL(10,2),
    scheduled_departure DATETIME,
    scheduled_arrival DATETIME,
    iot_device_id VARCHAR(100),
    status VARCHAR(50)
);

CREATE TABLE IoT_SensorReadings (
    reading_id INT AUTO_INCREMENT PRIMARY KEY,
    transport_id INT NOT NULL,
    reading_timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    speed_kmh DECIMAL(6,2),
    fuel_level_percent DECIMAL(5,2),
    engine_temperature_c DECIMAL(6,2),
    passenger_count INT,
    gps_latitude DECIMAL(10,6),
    gps_longitude DECIMAL(10,6),
    speed_status VARCHAR(50),
    FOREIGN KEY (transport_id) REFERENCES Transport(transport_id)
);

CREATE TABLE IoT_TransportAlerts (
    alert_id INT AUTO_INCREMENT PRIMARY KEY,
    transport_id INT NOT NULL,
    reading_id INT,
    alert_type VARCHAR(100),
    alert_severity VARCHAR(50),
    alert_message TEXT,
    alert_timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    resolved_status VARCHAR(50),
    FOREIGN KEY (transport_id) REFERENCES Transport(transport_id),
    FOREIGN KEY (reading_id) REFERENCES IoT_SensorReadings(reading_id)
);

CREATE TABLE IoT_TripSummary (
    summary_id INT AUTO_INCREMENT PRIMARY KEY,
    transport_id INT NOT NULL,
    trip_date DATE,
    total_distance_km DECIMAL(8,2),
    avg_speed_kmh DECIMAL(6,2),
    total_alert_count INT DEFAULT 0,
    trip_status VARCHAR(50),
    safety_score DECIMAL(5,2),
    FOREIGN KEY (transport_id) REFERENCES Transport(transport_id)
);

CREATE TABLE IoT_MaintenanceLog (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
    transport_id INT NOT NULL,
    alert_id INT UNIQUE,
    maintenance_date DATE,
    maintenance_type VARCHAR(100),
    description TEXT,
    technician_name VARCHAR(150),
    cost DECIMAL(10,2),
    FOREIGN KEY (transport_id) REFERENCES Transport(transport_id),
```

```
    FOREIGN KEY (alert_id) REFERENCES IoT_TransportAlerts(alert_id)
);
```

SQL Output 1: Database Tables Created

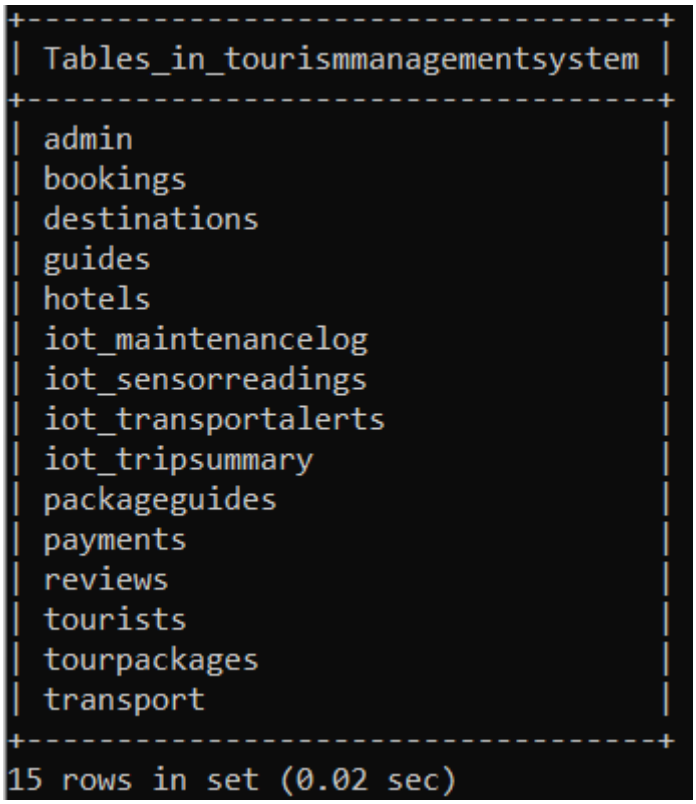


Figure 3 List of Tables Created in the Tourism Management System Database.

13. Database Verification and Display Queries

After creating the database, SQL commands were used to verify the structure and display records from all tables.

```
SHOW DATABASES;
USE TourismManagementSystem;
SHOW TABLES;

DESCRIBE Tourists;
DESCRIBE Destinations;
DESCRIBE Hotels;
DESCRIBE TourPackages;
DESCRIBE Bookings;
DESCRIBE Payments;
DESCRIBE Guides;
DESCRIBE PackageGuides;
DESCRIBE Reviews;
DESCRIBE Transport;
DESCRIBE IoT_SensorReadings;
DESCRIBE IoT_TransportAlerts;
DESCRIBE IoT_TripSummary;
DESCRIBE IoT_MaintenanceLog;

SELECT * FROM Tourists;
SELECT * FROM Destinations;
```

```
SELECT * FROM Hotels;
SELECT * FROM TourPackages;
SELECT * FROM Bookings;
SELECT * FROM Payments;
SELECT * FROM Guides;
SELECT * FROM PackageGuides;
SELECT * FROM Reviews;
SELECT * FROM Transport;
SELECT * FROM IoT_SensorReadings;
SELECT * FROM IoT_TransportAlerts;
SELECT * FROM IoT_TripSummary;
SELECT * FROM IoT_MaintenanceLog;
```

SQL Output 2: Structure of Tourists Table

```
+-----------------+--------------+------+-----+-------------------+-------------------+
| Field           | Type         | Null | Key | Default           | Extra             |
+-----------------+--------------+------+-----+-------------------+-------------------+
| tourist_id      | int          | NO   | PRI | NULL              | auto_increment    |
| full_name       | varchar(100) | NO   |     | NULL              |                   |
| gender          | varchar(10)  | YES  |     | NULL              |                   |
| date_of_birth   | date         | YES  |     | NULL              |                   |
| nationality     | varchar(50)  | YES  |     | NULL              |                   |
| email           | varchar(100) | NO   | UNI | NULL              |                   |
| phone           | varchar(20)  | YES  |     | NULL              |                   |
| passport_number | varchar(50)  | YES  |     | NULL              |                   |
| address         | varchar(255) | YES  |     | NULL              |                   |
| created_at      | timestamp    | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-----------------+--------------+------+-----+-------------------+-------------------+
10 rows in set (0.02 sec)
```

Figure 4 Structure of the Tourists Table.

SQL Output 3: Structure of Bookings Table

```
+------------------+---------------+------+-----+---------+----------------+
| Field            | Type          | Null | Key | Default | Extra          |
+------------------+---------------+------+-----+---------+----------------+
| booking_id       | int           | NO   | PRI | NULL    | auto_increment |
| tourist_id       | int           | YES  | MUL | NULL    |                |
| package_id       | int           | YES  | MUL | NULL    |                |
| booking_date     | date          | YES  |     | NULL    |                |
| number_of_people | int           | YES  |     | NULL    |                |
| total_amount     | decimal(10,2) | YES  |     | NULL    |                |
| booking_status   | varchar(30)   | YES  |     | Pending |                |
+------------------+---------------+------+-----+---------+----------------+
```

Figure 5 Structure of Bookings Table

SQL Output 4: Structure of Transport Table

```
+--------------------+---------------+------+-----+---------+----------------+
| Field              | Type          | Null | Key | Default | Extra          |
+--------------------+---------------+------+-----+---------+----------------+
| transport_id       | int           | NO   | PRI | NULL    | auto_increment |
| transport_type     | varchar(50)   | YES  |     | NULL    |                |
| company_name       | varchar(100)  | YES  |     | NULL    |                |
| departure_location | varchar(100)  | YES  |     | NULL    |                |
| arrival_location   | varchar(100)  | YES  |     | NULL    |                |
| departure_time     | datetime      | YES  |     | NULL    |                |
| arrival_time       | datetime      | YES  |     | NULL    |                |
| ticket_price       | decimal(10,2) | YES  |     | NULL    |                |
| iot_device_id      | varchar(50)   | YES  | UNI | NULL    |                |
| vehicle_number     | varchar(30)   | YES  |     | NULL    |                |
| iot_device_status  | varchar(20)   | YES  |     | Active  |                |
| iot_installed_date | date          | YES  |     | NULL    |                |
| last_ping_time     | datetime      | YES  |     | NULL    |                |
| is_iot_enabled     | tinyint(1)    | YES  |     | 1       |                |
+--------------------+---------------+------+-----+---------+----------------+
14 rows in set (0.01 sec)
```

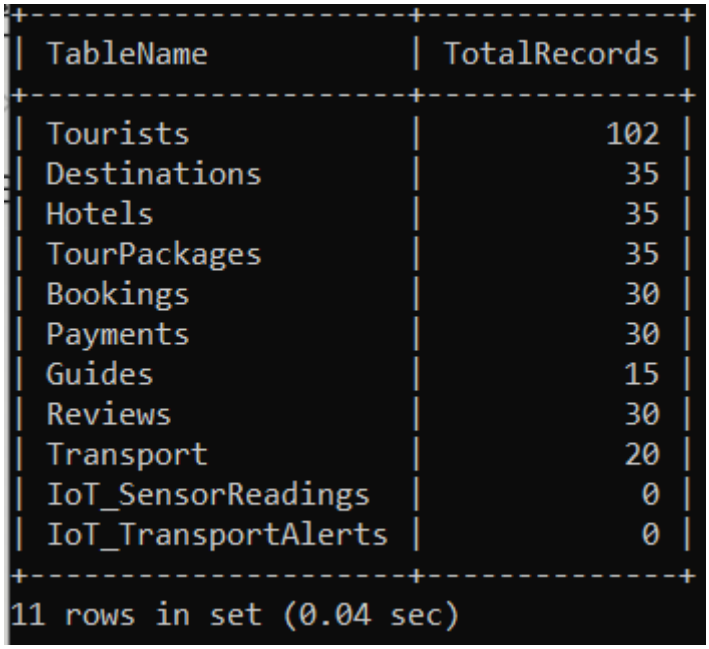
Figure 6 Structure of Transport Table

SQL Output 5: Structure of IoT Sensor Readings Table

```
+----------------------+---------------+------+-----+---------+----------------+
| Field                | Type          | Null | Key | Default | Extra          |
+----------------------+---------------+------+-----+---------+----------------+
| reading_id           | int           | NO   | PRI | NULL    | auto_increment |
| transport_id         | int           | NO   | MUL | NULL    |                |
| reading_timestamp    | datetime      | NO   |     | NULL    |                |
| gps_latitude         | decimal(10,7) | YES  |     | NULL    |                |
| gps_longitude        | decimal(10,7) | YES  |     | NULL    |                |
| gps_altitude_meters  | decimal(8,2)  | YES  |     | NULL    |                |
| gps_signal_strength  | varchar(10)   | YES  |     | NULL    |                |
| current_speed_kmh    | decimal(6,2)  | YES  |     | NULL    |                |
| max_allowed_speed    | decimal(6,2)  | YES  |     | NULL    |                |
| speed_status         | varchar(20)   | YES  |     | NULL    |                |
| cabin_temp_celsius   | decimal(5,2)  | YES  |     | NULL    |                |
| outside_temp_celsius | decimal(5,2)  | YES  |     | NULL    |                |
| engine_temp_celsius  | decimal(6,2)  | YES  |     | NULL    |                |
| fuel_level_percent   | decimal(5,2)  | YES  |     | NULL    |                |
| fuel_consumed_liters | decimal(7,2)  | YES  |     | NULL    |                |
| current_passengers   | int           | YES  |     | NULL    |                |
| max_capacity         | int           | YES  |     | NULL    |                |
| engine_status        | varchar(30)   | YES  |     | NULL    |                |
| engine_rpm           | int           | YES  |     | NULL    |                |
| battery_voltage      | decimal(5,2)  | YES  |     | NULL    |                |
| door_status          | varchar(20)   | YES  |     | NULL    |                |
+----------------------+---------------+------+-----+---------+----------------+
21 rows in set (0.00 sec)
```

Figure 7 Structure of IoT Sensor Readings Table

SQL Output 6: Total Records Stored in Main Tables



TableName	TotalRecords
Tourists	102
Destinations	35
Hotels	35
TourPackages	35
Bookings	30
Payments	30
Guides	15
Reviews	30
Transport	20
IoT_SensorReadings	0
IoT_TransportAlerts	0

11 rows in set (0.04 sec)

Figure 8 Total Records Stored in Main Tables

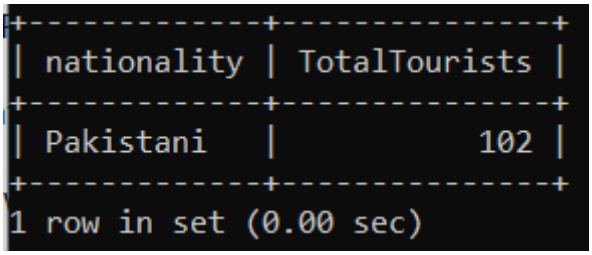
14. Important SQL Analysis Queries

Query 1: Tourist Nationality Distribution

This query shows how many tourists belong to each nationality. It helps to understand the main tourist groups in the system.

```
SELECT nationality,
COUNT(*) AS TotalTourists
FROM Tourists
GROUP BY nationality
ORDER BY TotalTourists DESC;
```

SQL Output 7: Tourist Nationality Distribution



nationality	TotalTourists
Pakistani	102

1 row in set (0.00 sec)

Figure 9 Tourist Nationality Distribution

Query 2: Most Booked Destinations

This query shows which destinations are booked the most. It joins destinations, tour packages, and bookings.

```
SELECT d.destination_name,
COUNT(b.booking_id) AS TotalBookings
FROM Destinations d
LEFT JOIN TourPackages tp ON d.destination_id = tp.destination_id
LEFT JOIN Bookings b ON tp.package_id = b.package_id
GROUP BY d.destination_name
ORDER BY TotalBookings DESC;
```

SQL Output 8: Most Booked Destinations

destination_name	TotalBookings
Fairy Meadows	1
Attabad Lake	1
Khunjerab Pass	1
Deosai Plains	1
Ratti Gali Lake	1
Saif ul Malook Lake	1
Babusar Top	1
Shogran	1

Figure 10 Most Booked Destinations

Maldives Beach Resort	1
Toli Pir	0
Makran Coastal Highway	0
Pakistan Monument	0
Rawal Lake	0
Minar e Pakistan	0

35 rows in set (0.00 sec)

Figure 11 Most Booked Destinations

Query 3: Most Popular Transport Routes

This query shows the most scheduled transport routes. It helps to find which routes are used more often.

```
SELECT departure_location,
arrival_location,
COUNT(*) AS TotalScheduledTrips
FROM Transport
GROUP BY departure_location, arrival_location
ORDER BY TotalScheduledTrips DESC;
```

SQL Output 10: Booking Status Summary

departure_location	arrival_location	TotalScheduledTrips
Karachi	Lahore	1
Lahore	Islamabad	1
Islamabad	Skardu	1
Karachi	Islamabad	1
Chilas	Fairy Meadows	1
Lahore	Multan	1
Islamabad	Gilgit	1
Gilgit	Hunza	1
Rawalpindi	Naran	1
Naran	Saif ul Malook	1
Islamabad	Quetta	1
Quetta	Gwadar	1
Karachi	Gwadar	1
Lahore	Rawalpindi	1
Islamabad	Dubai	1
Islamabad	Paris	1
Lahore	Tokyo	1
Islamabad	Male	1
Islamabad	Zurich	1
Peshawar	Kalam	1

20 rows in set (0.00 sec)

Figure 12 Booking Status Summary

Query 4: Booking Status Summary

This query shows the number of bookings in each booking status.

```
SELECT booking_status,
COUNT(*) AS TotalBookings
FROM Bookings
GROUP BY booking_status;
```

booking_status	TotalBookings
Confirmed	21
Pending	7
Cancelled	2

3 rows in set (0.00 sec)

Query 5: Revenue by Payment Method

This query shows total completed payment amount by payment method.

```
SELECT payment_method,
SUM(payment_amount) AS TotalRevenue
FROM Payments
WHERE payment_status = 'Completed'
GROUP BY payment_method
ORDER BY TotalRevenue DESC;
```

SQL Output 11: Revenue by Payment Method

```
+----------------+--------------+
| payment_method | TotalRevenue |
+----------------+--------------+
| Debit Card     |    667500.00 |
| Credit Card    |    320000.00 |
| Cash           |    298000.00 |
| JazzCash       |    237500.00 |
| Bank Transfer  |    220000.00 |
| EasyPaisa      |    191000.00 |
+----------------+--------------+
6 rows in set (0.191 sec)
```

Figure 13 Revenue by Payment Method

Query 6: IoT Transport Alerts by Severity

This query shows how many alerts exist in each severity level. It helps to understand transport safety issues.

```
SELECT alert_severity,
COUNT(*) AS TotalAlerts
FROM IoT_TransportAlerts
GROUP BY alert_severity
ORDER BY TotalAlerts DESC;
```

SQL Output 12: IoT Transport Alerts by Severity

```
+----------------+-------------+
| alert_severity | TotalAlerts |
+----------------+-------------+
| Medium         |           5 |
| High           |           5 |
| Low            |           2 |
| Critical       |           2 |
+----------------+-------------+
4 rows in set (0.627 sec)
```

Figure 14 IoT Transport Alerts by Severity

Query 7: Transport Utilization by Route

This query counts total trips for each route. It helps to analyze route utilization.

```
SELECT departure_location,
arrival_location,
COUNT(transport_id) AS TotalTrips
FROM Transport
```

```
GROUP BY departure_location, arrival_location
ORDER BY TotalTrips DESC;
```

SQL Output 13: Transport Utilization by Route

departure_location	arrival_location	TotalTrips
Karachi	Lahore	1
Lahore	Islamabad	1
Islamabad	Skardu	1
Karachi	Islamabad	1
Chilas	Fairy Meadows	1
Lahore	Multan	1
Islamabad	Gilgit	1
Gilgit	Hunza	1
Rawalpindi	Naran	1
Naran	Saif ul Malook	1
Islamabad	Quetta	1
Quetta	Gwadar	1
Karachi	Gwadar	1
Lahore	Rawalpindi	1
Islamabad	Dubai	1
Islamabad	Paris	1
Lahore	Tokyo	1
Islamabad	Male	1
Islamabad	Zurich	1
Peshawar	Kalam	1

20 rows in set (0.00 sec)

Figure 15 Transport Utilization by Route

15. Power BI Dashboard Objectives

Power BI was used to convert database records into visual insights. These charts make the data easier to understand. They also help management make better decisions.

Objective 1: Tourist Nationality Distribution

Purpose: To identify the distribution of tourists by nationality.

Recommended Visual: Clustered column chart or bar chart.

Power BI Fields:

X axis: nationality

Y axis: Count of tourist_id

Value name: TotalTourists

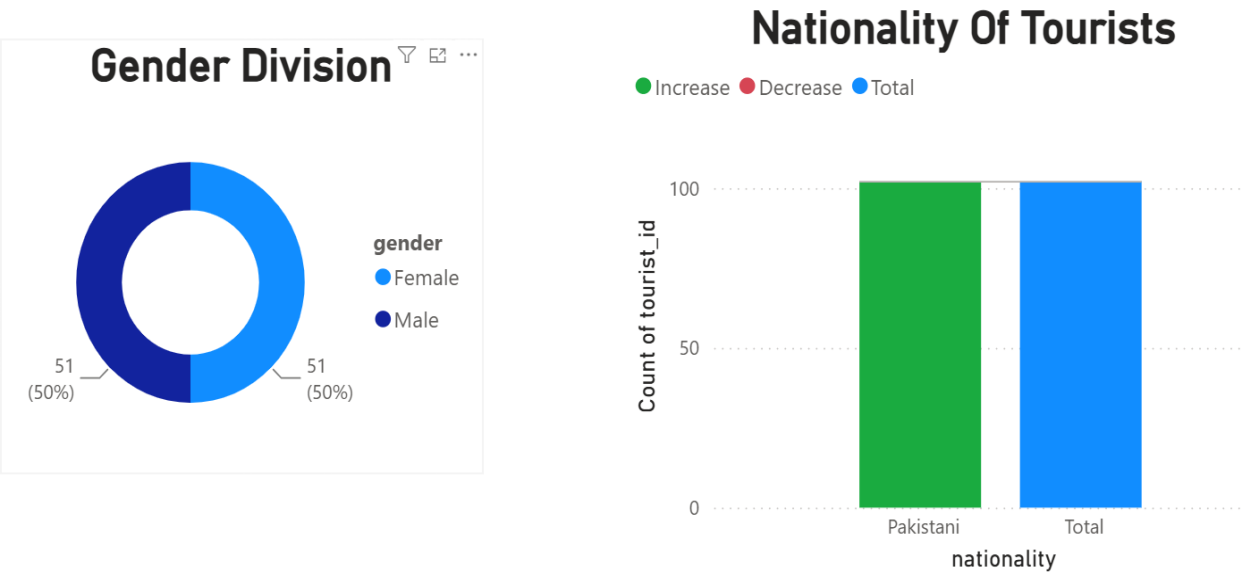


Figure 16 Tourist Nationality Distribution.

This chart shows how tourists are divided by nationality. It helps identify the countries or regions from where most tourists come. This information can support marketing, guide language planning, and service improvement.

Objective 1: Booking Status Analysis

The first Power BI objective is to analyze the booking status in the Tourism Management System. Booking status is important because it shows how many bookings are confirmed, pending, or cancelled. This helps the tourism management team understand the current condition of customer bookings.

The chart shows three booking categories. Confirmed bookings are the highest, with 21 bookings, which represent 70 percent of the total bookings. Pending bookings are 7, which represent 23.33 percent. Cancelled bookings are 2, which represent 6.67 percent.

This result shows that most customers have confirmed their bookings. This is a positive sign because it means the system has a strong number of completed booking decisions. The number of pending bookings is also important because these bookings still need follow up. The cancelled bookings are very low, which shows that only a small number of tourists cancelled their plans.

Overall, the booking status analysis helps in monitoring customer booking behavior. It also helps the tourism organization manage confirmed trips, follow pending bookings, and review cancelled bookings for better planning.

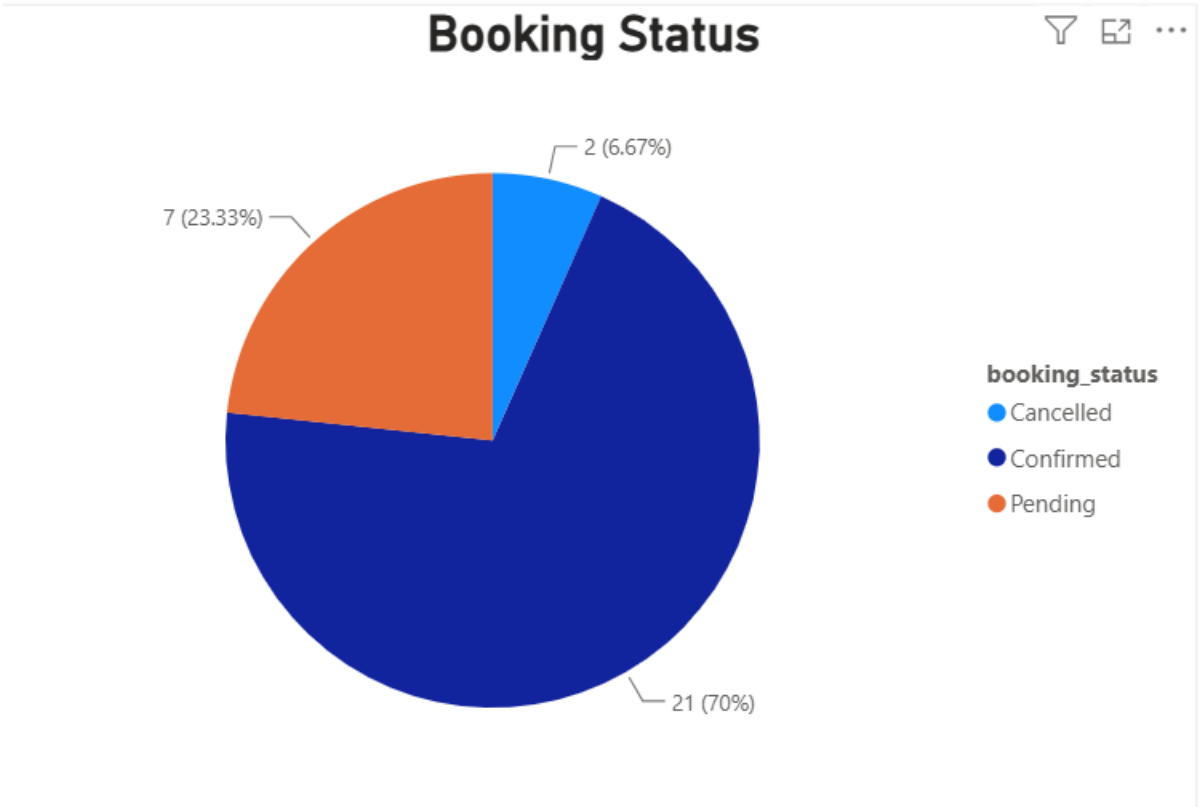


Figure 17 Booking Status Distribution

Objective 2: Payment Status Analysis

The second Power BI objective is to analyze the payment status in the Tourism Management System. Payment status is important because it shows how much payment amount is paid, pending, or refunded. This helps the tourism management team understand the financial condition of bookings.

The chart shows three payment categories. The paid amount is the highest, with approximately 3.9 million. The pending amount is approximately 1.2 million. The refunded amount is approximately 0.3 million.

This result shows that most of the payment amount has already been paid. This is a positive sign because it means a large portion of the booking revenue has been received. The pending amount still needs attention because it shows payments that are not completed yet. The refunded amount is lower, which means only a small portion of the total payment amount was returned.

Overall, the payment status analysis helps the tourism organization monitor revenue collection. It also helps identify pending payments and refunded amounts. This supports better financial tracking and booking management.

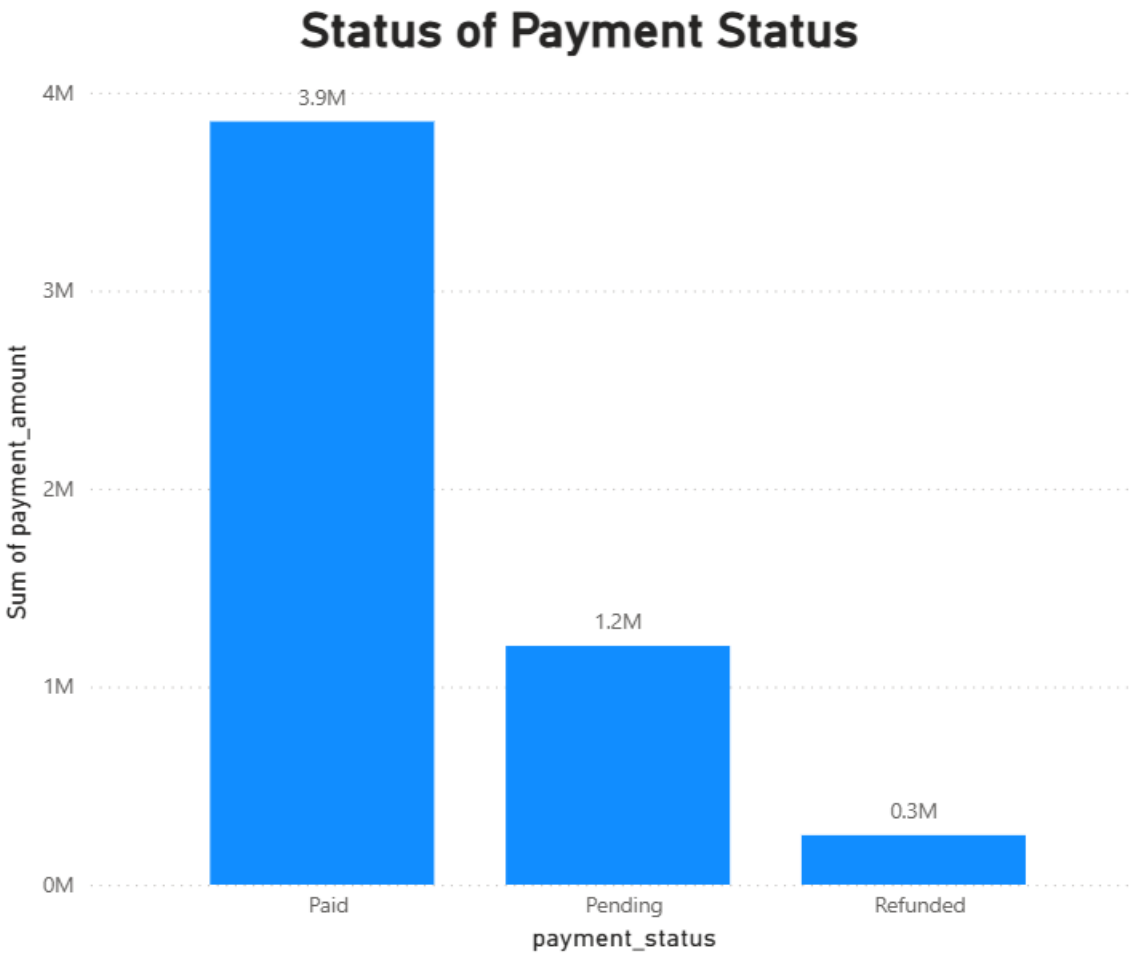


Figure 18 Payment Amount by Payment Status.

16. Discussion

The Tourism Management System database supports the main operations of a tourism organization. It stores tourist information, package details, destination records, booking data, payment records, guide assignments, reviews, and transport monitoring data in one structured system.

The ERD improves understanding of the database design. It clearly shows how the tables are connected. It also shows primary keys, foreign keys, and relationship cardinalities. This makes the database easier to understand and implement.

The EERD improves the design further. It adds advanced modelling concepts. Transport specialization makes the transport structure more flexible. GuideLanguages removes the multivalued language problem. PackageGuides solves the many to many relationship between packages and guides. IoT aggregation shows the monitoring process in a more meaningful way.

Normalization reduces duplicate data and improves consistency. It makes sure that data is stored in the correct table. It also helps avoid future errors when data is inserted, updated, or deleted.

Anomaly removal makes the system more reliable. It prevents duplicate guide assignments. It protects old booking amounts. It keeps payment records clear. It also ensures that trip summaries are generated in a controlled way.

The MySQL implementation confirms that the design can work as a real database. The verification queries help check the database structure. The analysis queries help extract useful results. Power BI then converts these results into charts. These charts support better decision making in tourism management.

17. Limitations

The project focuses on database design and analysis. It does not include a complete software application.

The IoT integration is represented at the database level only. Real time hardware connection is not included.

Online payment gateway integration is not included. The Payments table only stores payment records.

Power BI outputs depend on the sample data stored in the database. If the sample data is limited, the charts will also be limited.

Some derived fields need views, triggers, or generated columns for stronger accuracy. These include destination rating, speed status, average speed, and safety score.

18. Conclusion

This project successfully designed and implemented a Tourism Management System database with IoT based transport monitoring. The database provides a structured way to manage tourism records and transport monitoring records.

The ERD clearly represents the main entities and their relationships. The EERD improves the design by adding specialization, aggregation, associative entities, multivalued attribute handling, and derived attributes. Normalization improves data consistency and reduces repetition. Anomaly removal reduces common database problems such as insertion, update, deletion, duplication, and derived data issues.

The MySQL implementation confirms that the database can be created and used practically. The verification queries help check the tables. The analysis queries provide useful information from the database. Power BI visualizations make the data easier to understand and support better decision making.

Overall, this project provides a strong database foundation for a tourism management system. In the future, it can be extended with a web application, mobile application, live IoT sensors, online payment gateway, and advanced dashboard features.

19. References

Connolly, T., and Begg, C. (2015). Database Systems: A Practical Approach to Design, Implementation, and Management. Pearson Education.

Elmasri, R., and Navathe, S. (2016). Fundamentals of Database Systems. Pearson Education.

Ramakrishnan, R., and Gehrke, J. (2003). Database Management Systems. McGraw Hill.

Chen, P. P. (1976). The Entity Relationship Model: Toward a Unified View of Data. ACM Transactions on Database Systems.

Codd, E. F. (1970). A Relational Model of Data for Large Shared Data Banks. Communications of the ACM.

MySQL Documentation. MySQL 8.0 Reference Manual.

Microsoft Documentation. Power BI Documentation.